

COMPUTATIONAL METHODS IN FINANCE

Computational project 2

Author:

Cristóvão Matos (140686)

crisovao.matos@tecnico.ulisboa.pt

2025/2026 – 2nd Semester

2026/06/12

1 Introduction

This project explores the use of pseudo-random, quasi-random and stochastic simulation methods in computational finance. It begins with the generation of random variables and low-discrepancy points, then uses these samples in Monte Carlo and quasi-Monte Carlo integration, and finally applies numerical methods for stochastic differential equations.

We first implement routines to generate samples from the uniform distribution $U([0, 1])$, samples from the standard normal distribution $N(0, 1)$, and Halton nodes in $[0, 1]^2$. The uniform samples are generated with a linear congruential generator, while the normal samples are generated using the acceptance–rejection method. The quality of the generated samples is assessed by comparing empirical means, variances and histograms with the corresponding theoretical distributions.

The generated points are then used to approximate the area of the sets

$$A_\alpha = \{(x, y) \in [0, 1]^2 : f(x, y) < \alpha\}, \quad \alpha \in [-1, 1],$$

where

$$f(x, y) = \sin(10(x^2 - \sin(3y))).$$

The area $|A_\alpha|$ is written as an integral of an indicator function over the unit square and estimated using both Monte Carlo and quasi-Monte Carlo methods.

The final stage of the project focuses on stochastic differential equations. General MATLAB routines are implemented for the Euler–Maruyama and Milstein methods. These routines are then applied to the geometric Brownian motion

$$dS(t) = \mu S(t) dt + \sigma S(t) dB(t),$$

using the parameters specified in the project statement. Since this equation has an explicit solution, the numerical approximations can be compared directly with the exact path, and the strong and weak convergence orders of both methods can be estimated.

2 Random Number Generation

We start by constructing routines for generating pseudo-random and quasi-random samples. These routines are used later in the Monte Carlo and quasi-Monte Carlo simulations, so reproducibility is important. For this reason, the pseudo-random routines were implemented using an explicit seed.

Three types of samples were generated. First, pseudo-random realisations of the uniform distribution on $[0, 1]$ were obtained using a linear congruential generator. Second, realisations of the standard normal distribution were generated using the acceptance–rejection method. Finally, deterministic low-discrepancy points in the square $[0, 1] \times [0, 1]$ were generated using the Halton sequence.

2.1 Linear Congruential Generator

The basic source of pseudo-random numbers used in the project is the linear congruential generator. Starting from an initial seed m_0 , the sequence is defined recursively by

$$m_{i+1} = Am_i + B \pmod{M},$$

and the corresponding values in $[0, 1]$ are obtained by the normalisation

$$u_i = \frac{m_i}{M}.$$

In the implementation, we used the parameters

$$M = 2^{31} - 1, \quad A = 16807, \quad B = 0$$

which correspond to the classical Park–Miller multiplicative congruential generator. Since M is prime and $A = 16807$ is a primitive root modulo M , every nonzero seed generates a sequence with the maximal period $M - 1 = 2^{31} - 2$. These parameters are a standard historical choice for simple reproducible simulations and are also the values adopted in the lecture notes [1–3].

The generated values u_i are then interpreted as realisations of a random variable uniformly distributed on $[0, 1]$.

The quality of the generated samples was first assessed by comparing their empirical mean and variance with the theoretical values

$$\mathbb{E}[U] = \frac{1}{2}, \quad \text{Var}(U) = \frac{1}{12} \approx 0.083333.$$

The results are shown in Table 1. As the sample size increases, the estimated mean and variance approach the theoretical values.

N	$\hat{\mu}$	$\hat{s}^2$
1 000	0.509250	0.086249
10 000	0.500899	0.082109
1 000 000	0.500332	0.083415

Table 1: Mean and variance estimators from samples of N pseudo-random numbers generated with the linear congruential generator. For $U([0, 1])$, the theoretical values are $\mu = 0.5$ and $\sigma^2 = 1/12 \approx 0.083333$.

Figure 1 compares the empirical density of the generated samples for $N = 1000$ and $N = 100000$. The histogram becomes more stable as N increases, and the empirical density becomes closer to the constant density of the uniform distribution on $[0, 1]$.

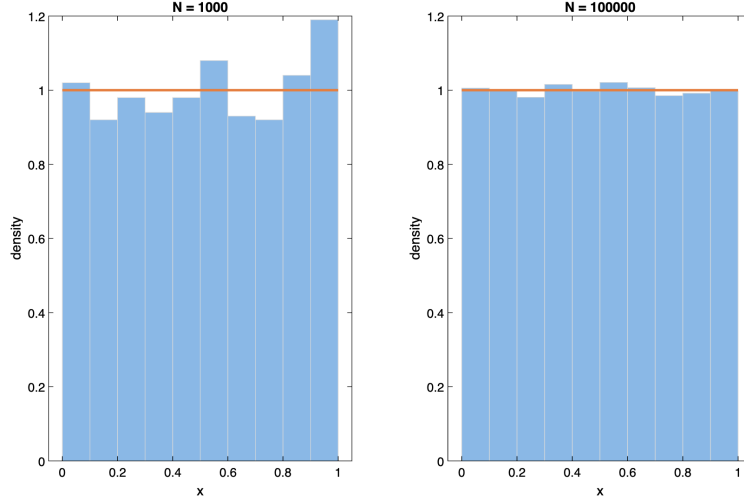


Figure 1: Histograms of pseudo-random samples generated with the linear congruential generator for $N = 1000$ and $N = 100000$, compared with the theoretical density of $U([0, 1])$.

2.2 Acceptance–Rejection Method

The acceptance–rejection method is useful when the target distribution cannot be simulated conveniently by closed-form inversion. The idea is to bound the target density by a scaled version of a proposal density that is easy to simulate. Candidate values are generated from the proposal distribution and accepted with a probability chosen so that the accepted values follow the target distribution.

The standard normal distribution was generated using the acceptance–rejection method described in Section 2.3.3 of the lecture notes [1]. The target density is

$$f(x) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{x^2}{2}\right), \quad x \in \mathbb{R}.$$

Following the example in the notes, the proposal density was chosen as the Cauchy density

$$g(x) = \frac{1}{\pi(1+x^2)}.$$

This distribution can be simulated explicitly by inversion. If $u_0 \sim U([0, 1])$, then

$$y = \tan\left(\pi u_0 - \frac{\pi}{2}\right)$$

has the Cauchy density g .

The acceptance–rejection method requires a constant $M > 0$ such that

$$f(x) \leq M g(x), \quad x \in \mathbb{R}.$$

For the densities above,

$$\frac{f(x)}{g(x)} = \sqrt{\frac{\pi}{2}} (1+x^2) \exp\left(-\frac{x^2}{2}\right).$$

The maximum of this ratio is attained at $x = \pm 1$, giving

$$M = \frac{f(1)}{g(1)} = \sqrt{\frac{2\pi}{e}}.$$

After generating a candidate y from the Cauchy proposal and an independent variable $u_1 \sim U([0, 1])$, the candidate is accepted whenever

$$u_1 M g(y) \leq f(y),$$

or equivalently,

$$u_1 \leq \frac{f(y)}{M g(y)}.$$

Geometrically, Mg forms an envelope above the target density f , and a candidate is accepted when the auxiliary uniform point falls below f within this envelope. Repeating the procedure produces accepted values with density f , that is, realisations of $N(0, 1)$. The theoretical acceptance probability is

$$\frac{1}{M} = \sqrt{\frac{e}{2\pi}} \approx 0.65774462.$$

The empirical mean and variance were compared with the theoretical values

$$\mathbb{E}[Z] = 0, \quad \text{Var}(Z) = 1.$$

The results are shown in Table 2. Although the estimates fluctuate for finite samples, they approach the theoretical values overall as the number of accepted realisations increases.

N	$\hat{\mu}$	$\hat{\sigma}^2$
500	0.023720	1.003275
5 000	0.011285	0.941084
50 000	0.002107	0.988023
500 000	0.002648	0.997094

Table 2: Mean and variance estimates from samples of N accepted realisations generated using the acceptance–rejection method. For $N(0, 1)$, the theoretical values are $\mu = 0$ and $\sigma^2 = 1$.

Figure 2 shows the corresponding histograms for increasing values of N . For smaller samples, the empirical density is visibly noisier, while for larger samples it follows the theoretical standard normal density more closely.

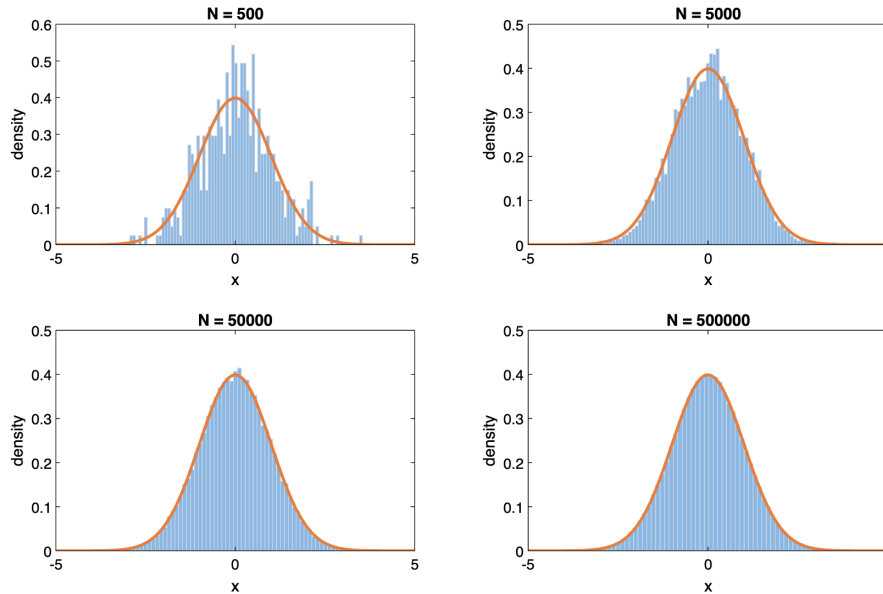


Figure 2: Histograms of samples generated from $N(0, 1)$ using the acceptance–rejection method for increasing values of N , compared with the theoretical standard normal density.

2.3 Halton Nodes

In addition to pseudo-random samples, quasi-random points were generated using the Halton sequence. The motivation, following the lecture notes [1], is to replace randomly generated points by points that are more evenly distributed over the integration region.

The construction is based on the radical inverse function. Given an integer n written in base b as

$$n = (d_k d_{k-1} \cdots d_1 d_0)_b,$$

or equivalently

$$n = d_k b^k + d_{k-1} b^{k-1} + \cdots + d_1 b + d_0,$$

the radical inverse function in base b is defined by

$$\phi_b(n) = \sum_{j=0}^k d_j b^{-j-1}.$$

That is,

$$\phi_b(n) = \frac{d_0}{b} + \frac{d_1}{b^2} + \cdots + \frac{d_k}{b^{k+1}}.$$

The Halton sequence in dimension m is obtained by using prime bases $p_1, \dots, p_m$. In dimension two, the first two prime numbers were used, so the nodes are

$$H_n = (\phi_2(n), \phi_3(n)), \quad n = 1, \dots, N.$$

These points are deterministic rather than pseudo-random. Their purpose is not to reproduce statistical independence, but to provide a more regular coverage of the unit square. In quasi-Monte Carlo methods, Halton nodes are an example of a low-discrepancy sequence: the proportion of points falling inside subregions of the unit square is designed to be close to the corresponding area. This more even coverage is the reason why low-discrepancy sequences can be useful for numerical integration [1, 3].

For low-discrepancy sequences, the discrepancy satisfies a bound of order

$$D_N = O\left(\frac{(\log N)^m}{N}\right),$$

where m is the dimension [3]. In this project $m = 2$, since the integration region is the unit square. This is a low-dimensional setting where Halton sequences are generally suitable, although their performance may deteriorate in higher dimensions [2].

Figure 3 compares 1000 pseudo-random points in $[0, 1] \times [0, 1]$ with 1000 Halton nodes. The pseudo-random points show visible clusters and empty regions, while the Halton nodes provide a more regular coverage of the unit square without forming a Cartesian grid.

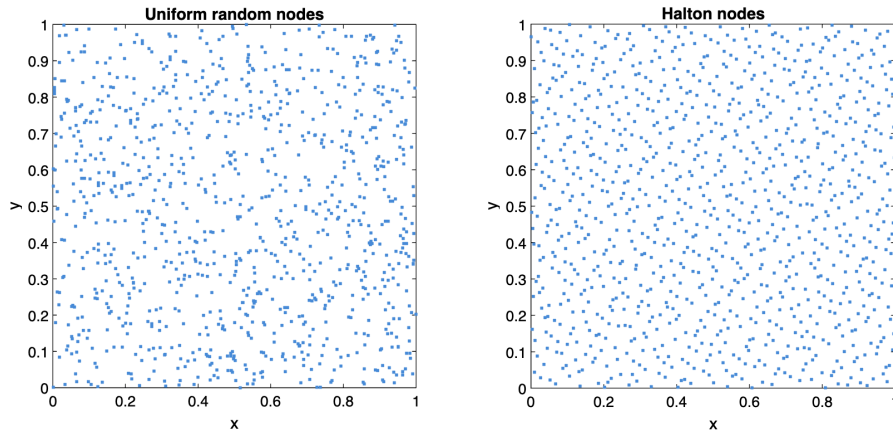


Figure 3: Comparison between 1000 pseudo-random points generated with the linear congruential generator and 1000 Halton nodes in the square $[0, 1] \times [0, 1]$.

3 Monte Carlo and Quasi-Monte Carlo Estimation of Areas

3.1 Problem Formulation

We consider the function

$$f(x, y) = \sin(10(x^2 - \sin(3y))), \quad (x, y) \in [0, 1]^2.$$

For each $\alpha \in [-1, 1]$, define

$$A_\alpha = \{(x, y) \in [0, 1]^2 : f(x, y) < \alpha\}.$$

The objective is to approximate the area function

$$\alpha \mapsto |A_\alpha|.$$

Let

$$g_\alpha(x, y) = \mathbf{1}_{\{f(x, y) < \alpha\}}.$$

Then the area can be written as

$$|A_\alpha| = \int_0^1 \int_0^1 g_\alpha(x, y) dx dy.$$

Thus, the problem is an integration problem over the unit square.

3.2 Monte Carlo and Quasi-Monte Carlo Estimators

Following the Monte Carlo integration formulation in the lecture notes [1], if

$$X = (X_1, X_2)$$

is uniformly distributed in

$$R = [0, 1]^2,$$

then, since $|R| = 1$,

$$|A_\alpha| = \int_R g_\alpha(x, y) dx dy = \mathbb{E}[g_\alpha(X)].$$

Therefore, for independent pseudo-random points

$$X_i = (X_{i,1}, X_{i,2}), \quad i = 1, \dots, N,$$

uniformly distributed in $[0, 1]^2$, the Monte Carlo estimator is

$$\hat{I}_{N,\alpha} = \frac{1}{N} \sum_{i=1}^N g_\alpha(X_i) = \frac{1}{N} \sum_{i=1}^N \mathbf{1}_{\{f(X_{i,1}, X_{i,2}) < \alpha\}}.$$

This is the proportion of generated points that fall inside A_α .

For the quasi-Monte Carlo estimator, the same average is computed using deterministic Halton nodes

$$z_i = (z_{i,1}, z_{i,2}), \quad i = 1, \dots, N.$$

Thus,

$$\tilde{I}_{N,\alpha} = \frac{1}{N} \sum_{i=1}^N g_\alpha(z_i) = \frac{1}{N} \sum_{i=1}^N \mathbf{1}_{\{f(z_{i,1}, z_{i,2}) < \alpha\}}.$$

The difference between the two estimators is therefore the choice of points: pseudo-random uniform points for Monte Carlo and Halton nodes for quasi-Monte Carlo.

The interval $[-1, 1]$ was discretised into 1000 equally spaced values,

$$-1 = \alpha_1 < \alpha_2 < \dots < \alpha_{1000} = 1.$$

For each α_j , we computed

$$\hat{I}_{N,\alpha_j} \quad \text{and} \quad \tilde{I}_{N,\alpha_j},$$

and used the points

$$(\alpha_j, \hat{I}_{N,\alpha_j}) \quad \text{and} \quad (\alpha_j, \tilde{I}_{N,\alpha_j})$$

to approximate the function

$$\alpha \mapsto |A_\alpha|.$$

3.3 Numerical Results

The numerical experiment for the full area curve was performed with $N = 10^6$ points for both the Monte Carlo and quasi-Monte Carlo estimators. Figure 4 shows the estimated function $\alpha \mapsto |A_\alpha|$.

The estimated curve is non-decreasing, as expected. Indeed, if $\alpha_1 < \alpha_2$, then $A_{\alpha_1} \subseteq A_{\alpha_2}$, and therefore $|A_{\alpha_1}| \leq |A_{\alpha_2}|$. The boundary behaviour is also consistent with the definition of A_α : for α close to -1 , the estimated area is close to zero, while for α close to 1, the estimated area approaches one.

The Monte Carlo and quasi-Monte Carlo estimates are almost indistinguishable at the scale of Figure 4. To make the comparison more quantitative, we also considered the representative value $\alpha = 0.5$. Since no closed-form value is available for $|A_{0.5}|$, a high-resolution quasi-Monte Carlo estimate using $N_{\text{ref}} = 5 \times 10^6$ Halton nodes was used as a reference value. This gave $I_{\text{ref}} \approx 0.73015440$.

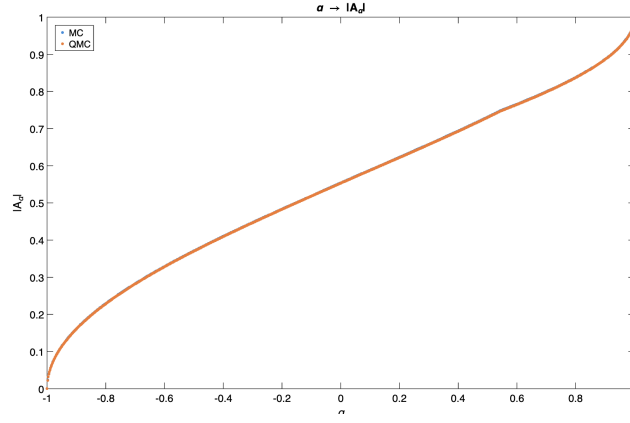


Figure 4: Monte Carlo and quasi-Monte Carlo approximations of the function $\alpha \mapsto |A_\alpha|$, using $N = 10^6$ points and 1000 equally spaced values of α in $[-1, 1]$.

Table 3 compares the Monte Carlo and quasi-Monte Carlo estimates for increasing values of N .

N	$\hat{I}_N$	$ I_{\text{ref}} - \hat{I}_N $	$\tilde{I}_N$	$ I_{\text{ref}} - \tilde{I}_N $
1 000	0.73700000	6.8456×10^{-3}	0.72200000	8.1544×10^{-3}
10 000	0.72500000	5.1544×10^{-3}	0.73060000	4.4560×10^{-4}
100 000	0.73166000	1.5056×10^{-3}	0.72990000	2.5440×10^{-4}
1 000 000	0.73095700	8.0260×10^{-4}	0.73014400	1.0400×10^{-5}

Table 3: Monte Carlo and quasi-Monte Carlo estimates of $|A_{0.5}|$, compared with the reference value $I_{\text{ref}} \approx 0.73015440$ obtained with 5×10^6 Halton nodes.

The results show that both methods approach the reference value as N increases. For $N = 1000$, the Monte Carlo estimate happens to be slightly closer to the reference value, which can occur for a single realisation and a discontinuous indicator integrand. For larger values of N , the quasi-Monte Carlo estimates are more accurate in this experiment, reflecting the more regular coverage of the integration region by the Halton nodes.

4 Numerical Methods for Stochastic Differential Equations

We implemented numerical routines for a general scalar stochastic differential equation of the form

$$dX(t) = a(t, X(t))dt + b(t, X(t))dB(t), \quad 0 < t \leq T,$$

with initial condition

$$X(0) = x_0.$$

The interval $[0, T]$ is discretised into equally spaced points

$$t_i = ih, \quad i = 0, 1, \dots, N, \quad h = \frac{T}{N}.$$

The objective is to compute approximations

$$X_i \approx X(t_i), \quad i = 0, 1, \dots, N.$$

Following the lecture notes [1], the integral form over one time step is

$$X(t_{i+1}) = X(t_i) + \int_{t_i}^{t_{i+1}} a(s, X(s)) ds + \int_{t_i}^{t_{i+1}} b(s, X(s)) dB(s).$$

Numerical methods are obtained by approximating the deterministic and stochastic integrals in this expression. The Brownian increments are generated as

$$\Delta B_i = B(t_{i+1}) - B(t_i) = \sqrt{h} Z_i, \quad Z_i \sim N(0, 1),$$

where the standard normal variables are generated using the routine developed in the random number generation section.

4.1 Euler–Maruyama Method

The Euler–Maruyama method is the stochastic analogue of the explicit Euler method for ordinary differential equations. It is obtained by approximating the drift and diffusion coefficients by their values at the left endpoint

t_i . The scheme is

$$X_{i+1} = X_i + a(t_i, X_i)h + b(t_i, X_i)\Delta B_i, \quad i = 0, 1, \dots, N-1.$$

Here, X_i denotes the numerical approximation of $X(t_i)$.

The implemented routine takes as inputs the functions $a(t, x)$ and $b(t, x)$, the initial condition x_0 , the final time T , the number of time steps N , and the seed used to generate the Brownian increments. It returns the time grid, the numerical approximation, the Brownian increments and the corresponding Brownian path.

When $b \equiv 0$, the stochastic term disappears and the method reduces to the usual explicit Euler method for ordinary differential equations [1]. In the stochastic case, both the exact solution and the numerical approximation depend on the Brownian realisation. Therefore, for a pathwise comparison, the same Brownian path must be used in the exact and numerical solutions.

Under suitable regularity assumptions, Euler–Maruyama has strong convergence order $1/2$ and weak convergence order 1 [1, 3]. The strong order refers to the pathwise terminal error, while the weak order refers to the error in expectations or other functionals of the terminal value.

4.2 Milstein Method

The Milstein method improves the Euler–Maruyama approximation by adding the first stochastic Taylor correction term. Assuming that the diffusion coefficient $b(t, x)$ is differentiable with respect to x , the method is

$$X_{i+1} = X_i + a(t_i, X_i)h + b(t_i, X_i)\Delta B_i + \frac{1}{2}b(t_i, X_i)b'(t_i, X_i)((\Delta B_i)^2 - h), \quad i = 0, 1, \dots, N-1,$$

where

$$b'(t, x) = \frac{\partial b}{\partial x}(t, x).$$

The additional term

$$\frac{1}{2}b(t_i, X_i)b'(t_i, X_i)((\Delta B_i)^2 - h)$$

comes from the stochastic Taylor expansion and accounts for the variation of the diffusion coefficient with respect to the state variable. This is the term that distinguishes Milstein from Euler–Maruyama.

The implemented Milstein routine is therefore similar to the Euler–Maruyama routine, but it also requires $b'(t, x)$ as an input. Under the usual regularity assumptions, Milstein has strong and weak convergence orders equal to 1 [1, 3]. This higher strong order is the main reason for comparing it with Euler–Maruyama in the geometric Brownian motion experiment.

Both routines were implemented for a general scalar SDE. They can therefore be applied once the coefficient functions $a(t, x)$, $b(t, x)$, and, for Milstein, $b'(t, x)$, are specified. In the next section, they are applied to the geometric Brownian motion SDE.

5 Geometric Brownian Motion Simulations

We now apply the Euler–Maruyama and Milstein methods to the geometric Brownian motion

$$dS(t) = \mu S(t) dt + \sigma S(t) dB(t), \quad 0 < t \leq T,$$

with parameters

$$S(0) = 10, \quad T = 1, \quad \mu = 0.6, \quad \sigma = 0.25.$$

This equation admits the exact solution

$$S(t) = S(0) \exp \left(\left(\mu - \frac{\sigma^2}{2} \right) t + \sigma B(t) \right).$$

Therefore, the numerical approximations can be compared directly with the exact solution along the same Brownian path.

5.1 Pathwise Comparison with the Exact Solution

For the pathwise comparison, we used a single realisation of the Brownian motion and step size

$$h = 0.001.$$

The same Brownian increments were used in the exact solution, the Euler–Maruyama approximation and the Milstein approximation. This ensures that the comparison reflects the numerical discretisation error and not differences caused by different Brownian paths.

For this SDE, the drift and diffusion coefficients are

$$a(t, S) = \mu S, \quad b(t, S) = \sigma S,$$

and the derivative required in the Milstein method is

$$\frac{\partial b}{\partial S}(t, S) = \sigma.$$

Thus, the Euler–Maruyama scheme becomes

$$S_{i+1} = S_i + \mu S_i h + \sigma S_i \Delta B_i,$$

while the Milstein scheme becomes

$$S_{i+1} = S_i + \mu S_i h + \sigma S_i \Delta B_i + \frac{1}{2} \sigma^2 S_i ((\Delta B_i)^2 - h).$$

Figure 5 shows the exact solution and the two numerical approximations. For the small step size used in this simulation, both methods track the exact path closely. The Milstein approximation is expected to provide a more accurate pathwise approximation because it includes the additional correction term involving $(\Delta B_i)^2 - h$.

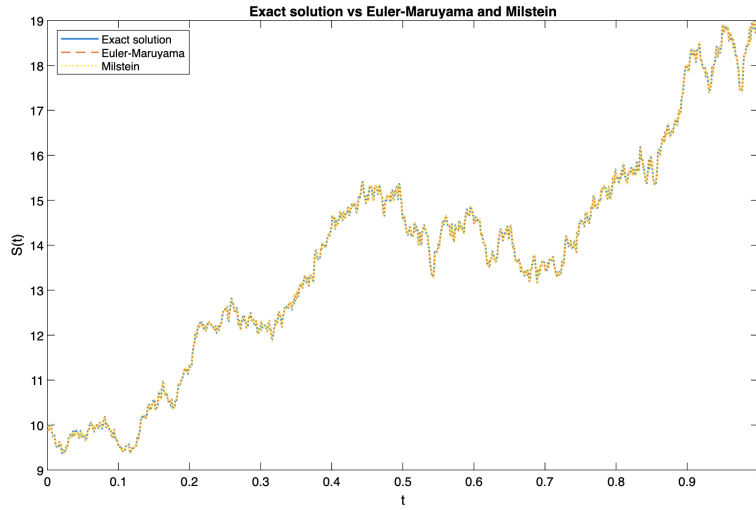


Figure 5: Pathwise comparison between the exact solution of the geometric Brownian motion and the Euler–Maruyama and Milstein approximations, using $h = 0.001$ and the same Brownian realisation.

5.2 Strong and Weak Convergence

We next estimate the strong and weak convergence orders of the Euler–Maruyama and Milstein methods. The time steps considered were

$$h_i = 0.005 \left(\frac{1}{2} \right)^i, \quad i = 0, 1, 2, 3,$$

that is,

$$h \in \{0.005, 0.0025, 0.00125, 0.000625\}.$$

For each value of h , we simulated

$$M = 10^6$$

independent Brownian paths. The strong error was estimated by

$$\mathbb{E} [|S(T) - S_N|],$$

where $S(T)$ is the exact terminal value and S_N is the numerical approximation at time T . This measures the average pathwise distance between the exact and numerical terminal values.

The weak error was estimated by

$$|\mathbb{E}[S(T) - S_N]| = |\mathbb{E}[S(T)] - \mathbb{E}[S_N]|.$$

This measures the error in the expected terminal value rather than the error along each individual path.

In the convergence experiment, only the terminal values were required. Therefore, the Euler–Maruyama and Milstein recursions were implemented directly in multiplicative form for the geometric Brownian motion

and accumulated through logarithms for numerical stability. This is equivalent to applying the corresponding one-step schemes to the GBM dynamics, but avoids storing all intermediate paths.

If the error behaves approximately as

$$\text{Error}(h) \approx Ch^\gamma,$$

then taking logarithms gives

$$\log(\text{Error}(h)) \approx \log(C) + \gamma \log(h).$$

The convergence order γ was estimated as the slope of the line fitted to the log-log plot of the error against h .

5.3 Numerical Results

The numerical errors obtained for the four time steps are reported in Table 4.

h	N	Strong EM	Strong Milstein	Weak EM	Weak Milstein
0.005000	200	4.7877×10^{-2}	1.7911×10^{-2}	1.6384×10^{-2}	1.6384×10^{-2}
0.002500	400	3.2992×10^{-2}	8.9628×10^{-3}	8.2350×10^{-3}	8.1929×10^{-3}
0.001250	800	2.3007×10^{-2}	4.4827×10^{-3}	4.1122×10^{-3}	4.0978×10^{-3}
0.000625	1600	1.6147×10^{-2}	2.2420×10^{-3}	2.0407×10^{-3}	2.0499×10^{-3}

Table 4: Estimated strong and weak errors for the Euler–Maruyama and Milstein methods applied to the geometric Brownian motion.

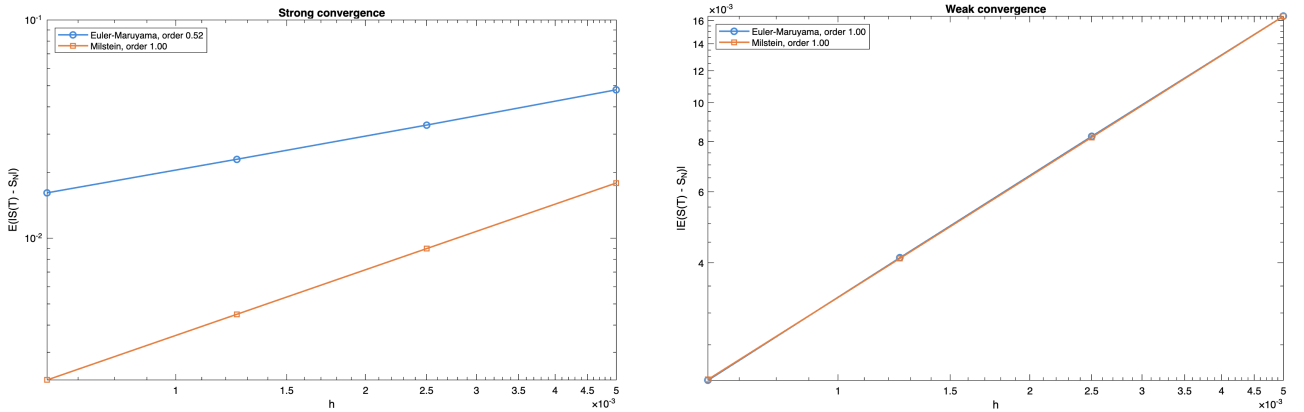
The estimated convergence orders are reported in Table 5.

Method	Strong order	Weak order
Euler–Maruyama	0.5224	1.0017
Milstein	0.9994	0.9995

Table 5: Estimated strong and weak convergence orders for the Euler–Maruyama and Milstein methods applied to the geometric Brownian motion.

These values are consistent with the expected theoretical behaviour. Euler–Maruyama has strong order approximately 1/2, while Milstein has strong order approximately 1. For weak convergence, both methods exhibit approximately first-order convergence in this experiment.

Figure 6 summarises the strong and weak convergence results. In Figure 6a, the Milstein error decreases approximately linearly with h in log-log scale, while the Euler–Maruyama error decreases more slowly, with order close to 1/2. In Figure 6b, both methods display approximately first-order convergence.



(a) Strong convergence errors for Euler–Maruyama and Milstein methods.

(b) Weak convergence errors for Euler–Maruyama and Milstein methods.

Figure 6: Strong and weak convergence results for the Euler–Maruyama and Milstein methods applied to the geometric Brownian motion. The estimated orders are approximately 0.52 and 1.00 for strong convergence, and approximately 1.00 for weak convergence for both methods.

Overall, the simulations are consistent with the expected numerical properties of the two schemes. The Euler–Maruyama method provides a simple first approximation and converges with strong order close to 1/2. The Milstein method improves the pathwise approximation by adding the stochastic correction term, leading to strong order close to 1. In weak convergence, both methods achieve approximately first-order accuracy for this test problem.

6 Discussion and Conclusions

The numerical results were consistent with the expected behaviour of the implemented methods. The linear congruential generator produced uniform samples whose empirical moments approached the theoretical values,

while the acceptance–rejection method generated samples increasingly close to the standard normal distribution as N increased.

Halton nodes provided a more regular coverage of the unit square than pseudo-random points. In the area estimation problem, Monte Carlo and quasi-Monte Carlo produced very similar curves for $N = 10^6$. For the representative value $\alpha = 0.5$, the quantitative comparison showed that quasi-Monte Carlo achieved markedly smaller errors for the larger sample sizes.

For geometric Brownian motion, both Euler–Maruyama and Milstein followed the exact path closely. The estimated convergence orders agreed with theory: Euler–Maruyama had strong order close to $1/2$, Milstein had strong order close to 1, and both methods showed approximately first-order weak convergence. Overall, the simulations provide numerical evidence that the implemented MATLAB routines behave as expected.

References

- [1] Pedro R. S. Antunes. A short introduction to computational methods in finance, 2026. Lecture notes, Instituto Superior Técnico.
- [2] James E. Gentle. *Random Number Generation and Monte Carlo Methods*. Statistics and Computing. Springer, New York, 2 edition, 2003.
- [3] Rüdiger U. Seydel. *Tools for Computational Finance*. Universitext. Springer, Berlin Heidelberg, 4 edition, 2009.